

CSCE 642 DEEP REINFORCEMENT LEARNING · MODULE 06 · LECTURE NOTES

Actor-Critic and Trust Regions

Both halves, and the default algorithm.

MODULE OUTCOMES

- Explain actor-critic and what each component contributes.
- Explain generalised advantage estimation.
- Explain why a trust region is needed and what TRPO computes.
- Explain PPO and why it became the default.
- Implement PPO correctly, including the details that matter.

1 Actor-critic

ACTOR-CRITIC IS A POLICY GRADIENT WITH A LEARNED BASELINE

The actor is the policy, updated by the policy gradient of Module 05. The critic is a value function, used as the baseline (Module 05 §2). There is nothing more to the idea than that, and framing it this way avoids treating it as a third family of methods.

So the critic reduces the actor's gradient variance, and the actor generates the on-policy data the critic learns from — a mutual dependence which is also why both can fail together.

And the critic enables bootstrapping — using $r + \gamma V(s')$ in place of the full episode return — **which reduces variance further and introduces bias,** which is **Module 03 §1's trade exactly**, now with the dial exposed as GAE's λ .

So actor-critic reintroduces the bootstrapping leg of the deadly triad (Module 04 §1) that pure policy gradients did not have. **Which is precisely why it is less stable than REINFORCE and more sample-efficient than it. Nothing in this field is free, and tracking which leg of the triad a method reintroduces is the most reliable way to predict how it will fail.**

$$\hat{A}_t = \sum_l (\gamma\lambda)^l \delta_{t+l}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t)$$

Generalised advantage estimation is an exponentially weighted average of every n-step advantage estimate at once. $\lambda = 0$ gives the one-step TD advantage (low variance, biased) and $\lambda = 1$ gives the Monte Carlo advantage (unbiased, high variance) — so λ is the bias-variance dial and it is continuous, which is Module 03 §3's n-step returns made smooth and differentiable in the parameter. $\lambda \approx 0.95$ with $\gamma \approx 0.99$ are the usual defaults and they are good ones, which is unusual enough in this field to be worth noting. **And λ and γ are separate dials with separate jobs** — γ sets the effective horizon of the problem (CSCE 625 Module 10 §1) and λ sets how much you trust the critic — **which students routinely**

conflate.

2 Why a trust region

A LARGE POLICY UPDATE CAN BE UNRECOVERABLE

In supervised learning a bad gradient step raises the loss and the next step corrects it. The training data is unchanged by the mistake, so the correction is available.

In reinforcement learning a bad step changes the policy, which changes the data distribution the agent collects — and the agent may now be gathering experience from a region of the state space from which it cannot find its way back. **The mistake has destroyed the data that would have corrected it.**

So performance collapse is not symmetric with improvement, and it is frequently permanent within a training run. This is the 'learns, then collapses' symptom of Module 04 §4, and it is one of the two or three things that make deep reinforcement learning frustrating in a way supervised learning is not.

A fixed learning rate cannot prevent it, because the same step in parameter space produces very different changes in the policy depending on where in parameter space you are — near a saturated softmax a small weight change barely moves the distribution, and elsewhere it moves it completely. **Which is why the constraint has to be on the policy rather than on the parameters,** and that single observation is the motivation for everything in §2 and §3.

TRPO: maximise the surrogate objective subject to
 $KL(\pi_{old} || \pi_{new}) \leq \delta$

- a constraint in POLICY space, not parameter space, which is the entire point (see the callout).
- solved by a conjugate-gradient step using the Fisher information matrix, which is the natural gradient -- CSCE 669 Module 07's second-order idea, with the metric supplied by the policy's own geometry.
- Principled, with provably monotone improvement under its assumptions, and a genuine pain to implement.

PPO: approximate the same thing with CLIPPING.

```
ratio r = pi_new(a|s) / pi_old(a|s)
objective = min( r * A, clip(r, 1-e, 1+e) * A )
```

If the ratio moves beyond $[1-e, 1+e]$ in the direction that would IMPROVE the objective, the gradient is CLIPPED TO ZERO -- the update simply stops pushing further.

If it moves in the direction that makes the objective WORSE, it is NOT clipped -- so the policy can always be pulled back from a bad place.

THAT ASYMMETRY IS THE DESIGN, and most explanations omit it. $e = 0.2$ typically.

PPO IS THE DEFAULT because it is about twenty lines, uses an ordinary first-order optimiser, allows several epochs of updates per collected batch (Module 05 section 3's partial data reuse), and performs close to TRPO on most tasks.

3 Implementing PPO

- **Observation and advantage normalisation. Running normalisation of observations is worth more than most algorithmic choices** — CSCE 633 Module 02's feature scaling and CSCE 669 Module 05 §2's conditioning argument, both arriving as the highest-value implementation detail in the algorithm.
- **Orthogonal weight initialisation, with a small gain on the policy head's final layer** — typically 0.01 — so that the initial policy is close to uniform rather than arbitrarily committed, which protects early exploration.
- **Learning rate annealing to zero over the course of training. Consistently helps**, and it is CSCE 669 Module 08 §2's noise-floor argument: the step must shrink for the policy to settle.
- **Value loss clipping, gradient norm clipping at 0.5, and a small entropy bonus** (Module 05 §4, Module 07 §3).
- **And correct handling of episode boundaries in the GAE computation. Bootstrapping $V(s')$ across a terminal state is a real bug** — the value after termination is zero, not whatever the critic says — **and it silently costs performance without producing any error**. It is the single most common PPO implementation bug and it is worth checking first.

THE IMPLEMENTATION-DETAIL FINDING, STATED PLAINLY

Careful replication studies have found that PPO's performance depends heavily on implementation details that are not described in the paper, and that some reported algorithmic advances over it disappear once those details are matched between the methods being compared.

That is a serious finding about the field's evidential standards rather than a minor caveat about one algorithm — it means a substantial fraction of the published comparison literature may be comparing implementations rather than ideas. **Module 12 develops this properly.**

The practical response is to compare against a strong, tuned baseline with the details present, rather than against a number reported in a paper whose implementation you have not read. **This is CSCE 633 Module 02's baseline discipline with a specific and well-documented reason attached.**

And to read a reference implementation rather than only the paper. CleanRL's single-file PPO is the right one, because nothing is hidden behind a framework abstraction and every detail in the list above is visible in sequence. **Reading it carefully once is worth more than reading three more papers.**

4 Choosing

Situation	Use	Why
A fast simulator and many parallel environments	PPO.	Simple, robust, and it scales well with parallel collection — which is what the large batches of Module 05 §4 require.

Situation	Use	Why
Samples are expensive	SAC (Module 07).	Off-policy, so a replay buffer reuses every transition many times.
Discrete, small action space, and many samples available	DQN and its variants (Module 04).	Replay efficiency, and the max over actions is cheap.
Very few samples available at all	Model-based methods (Module 08).	Orders of magnitude fewer environment steps, at the cost of model-exploitation risk.
A fixed logged dataset and no interaction	Offline reinforcement learning (Module 09).	PPO cannot use it at all, because it is on-policy and the data is not from its policy.

PPO is the default because it is robust, not because it is best. It rarely wins on sample efficiency and it rarely fails outright — which is the right trade when a simulator is cheap and your time is not, and it is the wrong trade when environment steps are the scarce resource. Stating it as 'robust, not best' is the honest characterisation, and it is how practitioners actually describe it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Schulman et al. — Proximal Policy Optimization Algorithms (free)	PPO. Short, and read it alongside §3's detail list rather than alone.
Schulman et al. — Trust Region Policy Optimization (free)	§2's principled version, with the monotone improvement argument that PPO approximates.
Engstrom et al. — Implementation Matters in Deep Policy Gradients (free)	§3's finding, measured — and the paper that should change how you read comparisons in this field.
CleanRL — ppo.py and the PPO implementation-details blog (free)	Every detail from §3, enumerated and benchmarked. The single most useful page in this course.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Implement advantage actor-critic and compare against REINFORCE with a baseline.
- 2 Implement GAE and sweep λ from 0 to 1. Plot performance and advantage variance.
- 3 Confirm $\lambda = 0$ and $\lambda = 1$ reduce to one-step TD and Monte Carlo.
- 4 Cause a policy collapse deliberately with a large learning rate, and confirm it does not recover.

- 5 **Implement PPO from scratch** and match CleanRL's CartPole performance.
- 6 **Remove the clipping** and report what happens.
- 7 **Remove the asymmetry** (clip both directions) and report the difference.
- 8 **Ablate each of §3's five details** and report the effect of each.
- 9 **Introduce the terminal-state bootstrapping bug** and measure the performance cost.
- 10 **Compare your PPO against a tuned reference** across five seeds, and report honestly.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What is actor-critic, and which triad leg does it reintroduce?
- 2 What does GAE's λ control, and how does it differ from γ ?
- 3 Why is a bad policy step worse than a bad supervised step?
- 4 Why must the constraint be in policy space?
- 5 What does TRPO compute, and what makes it hard?
- 6 State PPO's objective and explain the asymmetry.
- 7 Name five implementation details that determine PPO's performance.
- 8 What is the implementation-detail finding, and what should you do about it?
- 9 Give five situations and the algorithm for each.